When can you differentiate coupled simulation components separately?

A cheap test, and a cold plate that answers it

Tesseract Hackathon 2026 · Track: multi-physics & coupled systems · Author: @TAUIL-Abd-Elilah · source · Apache-2.0

Abstract

Composing differentiable simulation components across a boundary is expensive: it demands that each component expose derivatives, and that the composition solve an implicit system rather than a chain. A practitioner facing that cost reasonably asks whether the shortcut — differentiating each component in isolation and multiplying the pieces together — is good enough.

We build a natural-convection cold plate with three active Tesseracts and a drop-in thermal backend: three implementation languages and four derivative stacks across the repository. Three findings. **First**, the backends are genuinely interchangeable: replacing a JAX-autodiffed solver with a Fortran one differentiated by an Enzyme compiler pass leaves the end-to-end gradient unchanged to 5.3×10^{-12}. **Second**, the cost of skipping composition is wildly regime-dependent — from 0.01% to 86% relative error on the same code — and nothing in the forward solution indicates which regime you are in. **Third**, spectral radius of the fixed-point Jacobian, $\rho(\Phi\_T)$, is insufficient by itself (we exhibit a fixed-state case where it is constant while the error varies 136-fold), whereas the objective-aware residual $\gamma = \|\Phi\_T^T g\|/\|g\|$ — one vector-Jacobian product — tracks it (0.995 against 0.825 in log-log correlation; 0.994–0.997 under family holdout). Removing the physics entirely, γ tracks the error at **0.989 against 0.691 for ρ across 2,377 randomly generated coupled fixed points**, winning in every structural family and for nonlinear loops as well as linear ones — with one boundary we report rather than bury: that agreement is carried by attracting fixed points, and γ predicts poorly when the fixed point repels.

Two consequences we test rather than assert. The shortcut's failure is *modal*: it keeps the sign of every one of the fifty most influential design variables, which is why an optimiser driven by it still succeeds, while ranking those variables no better than chance (Spearman −0.011) — serviceable as a search direction, worthless as a sensitivity. And γ, being cheap, is usable as a budget: gating the optimiser on it removes 92% of the cross-boundary adjoint work while reaching the same design, and refuses correctly at the operating point where the shortcut carries 115% error.

Most importantly, the gradients change an action: under the same zero-net- material budget at strong coupling, cells selected by the composed sensitivity deliver **58% more realised cooling** than cells selected by the shortcut in a true coupled forward re-solve.

1. The problem

Two solvers that feed each other do not form a pipeline. In our case a Stokes–Brinkman flow solver and an advection–diffusion thermal solver are coupled in both directions: buoyancy makes temperature drive the flow, advection makes the flow drive temperature. The steady state is a fixed point,

$T^* = \Phi(T^*, \theta)$, $\Phi = \text{thermal}(\text{fluid}(T))$,

and the sensitivity of any objective $J(T^*)$ requires the implicit function theorem,

$dJ/d\theta = (\partial\Phi/\partial\theta)^T \lambda$, $(I - \Phi\_T)^T \lambda = g$, $g = dJ/dT$. (1)

The shortcut treats the loop as feed-forward and uses $\lambda_0 = g$. Its residual in the exact adjoint equation is

$r_0 = g - (I - \Phi\_T)^T\lambda_0 = \Phi\_T^T g$, $\lambda - \lambda_0 = (I - \Phi\_T^T)^{-1} r_0$. (2)

The first identity is exact whenever the derivatives exist; the second holds when the adjoint system is invertible. A Neumann expansion of the inverse would additionally require $\rho(\Phi\_T) < 1$ and is not used at our repelling headline state.

Relation to existing work

Almost everything here has been done before, and better, in one respect or another. Differentiable topology optimisation of thermo-fluidic devices: **TOFLUX** (Padmanabha et al., arXiv:2508.17564, 2025) is an open-source JAX framework covering thermo-fluidic coupling, FSI and non-Newtonian flow. Natural-convection heat-sink design: **Alexandersen et al.** (*Int. J. Heat Mass Transfer*, 2016, arXiv:1508.04596) solve the 3D Boussinesq problem with full Navier–Stokes at 40–330 million degrees of freedom; this work is 2D, Stokes and 96×96, and the branching structure it finds reproduces theirs qualitatively rather than adding to it. Sensitivity of fixed points under approximate linearisation: **Padway and Mavriplis** (*Numerical Algorithms*, 2021, arXiv:2104.02826) analyse tangent and adjoint problems linearised about non-stationary points. Our loop-cut λ_0 is a severe approximate adjoint, and $\Phi\_T^T g$ is its exact equation residual.

Three things are left. **Composition across genuinely heterogeneous components**, which those frameworks deliberately avoid — TOFLUX is one framework in one process because that is the sane way to build a framework — whereas here a hand-adjointed C++ solver, a compiler-differentiated Fortran solver, a JAX solver and a PyTorch model compose into one function, two of them interchangeably. **The demonstration that spectral radius alone is insufficient**, which we have not seen stated for this decision: $\rho(\Phi\_T)$ is the diagnostic a practitioner reaches for first and Section 5 shows it constant while the error moves 136-fold. And **γ as an operational check** — the analysis is standard, but making it a single VJP that runs as a pipeline assertion, and measuring what it actually predicts, is not.

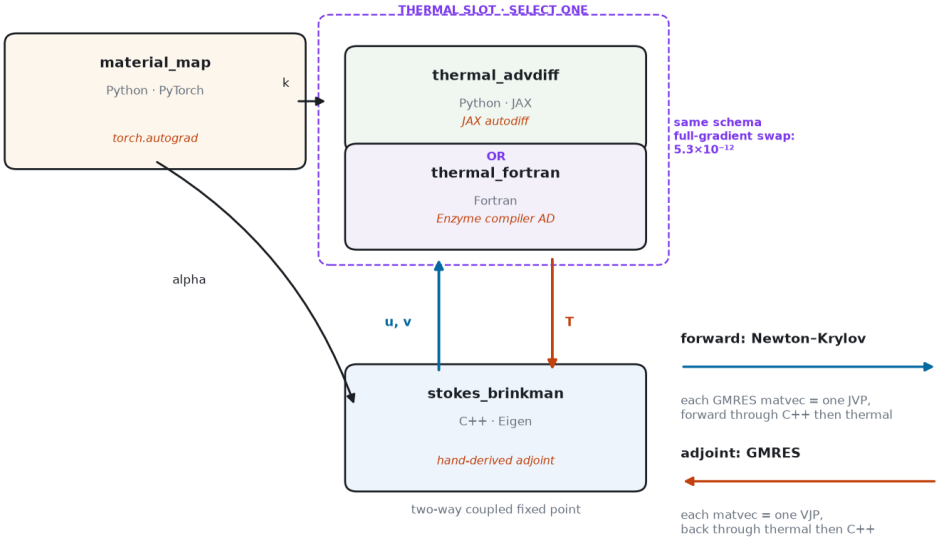
2. The composition

Three Tesseracts are active in a run; the thermal slot selects one of two backends. Across the repository there are three implementation languages and four ways of producing a derivative:

component	language	derivatives from
`stokes_brinkman`	C++ / Eigen	hand-derived discrete adjoint (no AD tool)
`thermal_advdiff`	Python / JAX	`jax.jvp` / `jax.vjp`
`thermal_fortran`	Fortran	Enzyme, an LLVM compiler pass
`material_map`	Python / PyTorch	`torch.autograd`

The fluid system is linear in w = (u,v,p), so `A(α) w = b(T)` and its exact JVP and VJP are extra solves against the same sparse LU — a transpose solve plus an analytic scatter, by hand. The Fortran component inverts that approach: flang emits LLVM IR, an Enzyme

pass differentiates it, and ∂R/∂T is recovered *exactly* from nine Enzyme JVPs by a 3×3 colouring (the stencil is five-point, so cells whose indices agree mod 3 never interact).



Three implementation languages · four derivative stacks · three served components per run.

The coupled adjoint exists only as a conversation between the active solvers — it cannot be assembled component by component. The thermal slot accepts either backend with no caller change, and the end-to-end gradient is unchanged to 5.3×10⁻¹².

> **The composition, and where derivative information travels.** Each run serves the material map, fluid solver and one implementation in the thermal slot. The design flows into the two-way fluid–thermal loop; the adjoint travels back up it. Each box-boundary arrow is a container round-trip, crossed once per Krylov matvec.

Both halves of the gradient are Krylov solves whose matvecs cross the container boundary: Newton–Krylov forward, applying $(\Phi_T - I)$ via JVPs, and GMRES for the adjoint of (1), applying $(I - \Phi_T)^T$ via VJPs. At our operating point the loop gain exceeds one, so Picard iteration provably cannot converge — Newton needs the linearisation invertible, not contractive. The JVP endpoints are therefore not a convenience; they are what makes the forward problem solvable.

A nonlinear component, and what the hand derivation costs. Modelling the flow as Stokes is itself an approximation, so the fluid block carries the convective acceleration behind a weight: `inertia = 0` is the infinite-Prandtl limit and reproduces every earlier result bitwise, `inertia = 1` is steady Navier–Stokes and makes the block nonlinear in w, solved by damped Newton.

The hand-derived adjoint survives intact. (u·∇)u is *bilinear* and involves neither α nor T, so every parameter scatter is unchanged and only the inverted operator moves from A to the Jacobian at the converged state, $J = A + \partial N/\partial w$. That is the practical argument for deriving an adjoint rather than reaching for a tool: the structure tells you which part a new nonlinearity touches, and it is a small part. All of it is checked against the autodiffed reference — forward < 10^{-8}, JVP and VJP < 10^{-7} through the Newton solve, and the adjoint identity to 10^{-8} — with two tests present solely to stop the rest passing vacuously, one asserting that inertia moves the solution and one that the inertial tangent differs from the Stokes tangent. The full composition still matches a coupled finite difference to 8.5×10^{-8} with the nonlinear block in place.

Having built it, we ask of this shortcut what the rest of the paper asks of loop-cutting. At the operating point used throughout — Ra = 3×10^4, mean density 0.5 — dropping inertia changes the design gradient by **0.002% in water and 0.017% in air**, cosine 1 to eight decimals (`inertia_study.py`). Brinkman drag is a linear sink on exactly the velocities the convective term feeds on, and at an rms speed of 0.4 that term sits orders of magnitude below the viscous one. So the Stokes limit used for every headline number is justified by measurement rather than by appeal to Pr → ∞ — and the general point stands again: a shortcut's safety is a property of the regime, not of the model.

Chain versus loop. The distinction matters and is often blurred. A *chain* — A feeds B feeds an objective — is differentiable by one sweep of the chain rule; a single `jax.grad` over wrapped components suffices and nothing is solved. This is a *loop*: the fluid solver's output is the thermal solver's input and vice versa, so no ordering makes one sweep sufficient. The steady state must be solved for, and its sensitivity requires a second, transposed solve whose operator exists only as an action realised by calling both components.

Why not `jax.custom_vjp`? The fair objection to any containerised differentiable pipeline is that one could attach a hand-written backward pass to a component and keep everything in one process. That is correct when the components are Python. Here the components are C++/Eigen and flang/Enzyme-built Fortran, whose build toolchains (LLVM 19, flang) do not belong inside a JAX process; the boundary is crossed thousands of times per solve in both directions, which is a conversation rather than a wrapped call; and because the contract is a schema, the two thermal implementations are substitutable with no change to the caller, which a `custom_vjp` written against one implementation is not.

Two claims are enforced by the build rather than asserted: neither the C++ nor the Fortran image can import `jax`, `torch`, `tensorflow`, `autograd` or `casadi`, and the Fortran library must contain `cosh` among its linked symbols — a function present in no source file, being Enzyme's generated derivative of the `tanh` in the Péclet weighting. Both are re-checkable from outside the build with `scripts/verify_integrity.sh`.

Components are interchangeable. `thermal_advdiff` and `thermal_fortran` implement the same equation behind the same schema. Swapping them (`compare_thermal_backends.py`) changes the component field by 7.1×10^{-16}, the JVP by 1.4×10^{-15}, the VJP by 4.3×10^{-15}, the converged coupled state by 4.8×10^{-12}, and **the end-to-end gradient by 5.3×10^{-12}**, cosine 1.000000000000. The gradient does not depend on which derivative technology produced it.

3. Validation

Classical critical Rayleigh number. Stokes flow is the infinite-Prandtl limit, which is the regime in which the classical onset result $Ra_c = 1707.762$ is derived, so it is the correct benchmark for this solver. (A Navier–Stokes benchmark such as de Vahl Davis is not: at Pr = 0.71 the inertia term we omit is not small, and disagreement would prove nothing.) At the conduction state the coupling loop *is* the linear stability operator, so onset occurs exactly where $\rho(\Phi_T) = 1$. Bisecting on Ra (`benchmark_critical_rayleigh.py`):

aspect ratio	1	2	4	8
Ra_c measured	2519.07	1970.84	1779.02	**1707.97**
excess	+47.5%	+15.4%	+4.2%	**+0.012%**

No-slip side walls stabilise a confined layer, so Ra_c must exceed the unbounded value and descend towards it as the box widens. It does, agreeing to four significant figures. One number checks the fluid solver, the thermal solver, the coupling between them and the loop-gain machinery simultaneously.

Order of accuracy. Richardson extrapolation on two independent grid trios gives observed order 1.87 and 1.83, with 0.02–0.05% error on the finest grid (`grid_convergence.py`).

Independent implementations agree. The composed pipeline reproduces a separately written monolithic reference to 1.5×10^{-12} on the converged coupled state, reached by a different nonlinear solver (5 Newton iterations against 21 Picard). Conduction-only energy balance closes to 5.3×10^{-15}.

The composed gradient is exact. A directional derivative agrees with central differences to 8.3×10^{-6}, holding at $\sim 10^{-5}$ across step sizes from 10^{-3} to 10^{-4} — that plateau is the differencing noise floor, since the fixed point is converged only to $\sim 10^{-10}$.

4. What the shortcut costs

We compare against the *charitable* shortcut: it uses the fluid solver's adjoint in full and differentiates the whole chain ρ → α → (u,v) → T, and is exact except that it treats the temperature entering buoyancy as constant. It errs only by ignoring the loop.

At a strongly coupled state (Ra = 3×10^4) it carries **86% relative error**, cosine 0.53 against the true gradient, and the **wrong sign on 33% of design variables**. At the states our optimiser actually visits it is 4–20% off with cosine above 0.98. Same code, same physics, two orders of magnitude difference — and J moves by only 13% across the range over which the gradient error grows from 3×10^{-6} to 0.86. **The forward solution gives no warning.**

Spatially, the error is not noise. At high coupling the exact gradient develops a coherent region of *positive* sensitivity — adding material there makes the chip hotter, because it obstructs the convection cell removing the heat — and the shortcut has no such region anywhere. The disagreement is one contiguous blob: an entire physical term, missing.

5. Predicting it

The obvious statistic fails. ρ(Φ_T) is the natural candidate and correlates respectably (0.825 with log error) when designs and Rayleigh numbers are varied together. But it is a worst case over all directions, and gain along directions the objective never excites costs nothing. Two states at the same Ra have ρ = 0.682 with 40% error and ρ = 0.377 with 79% error: it orders them backwards.

A constant cannot explain a variable. The decisive experiment holds the design, the Rayleigh number and hence the entire coupled state fixed, and varies only the objective. ρ(Φ_T) is then *identical by construction*, while g = dJ/dT differs (`objective_sweep.py`):

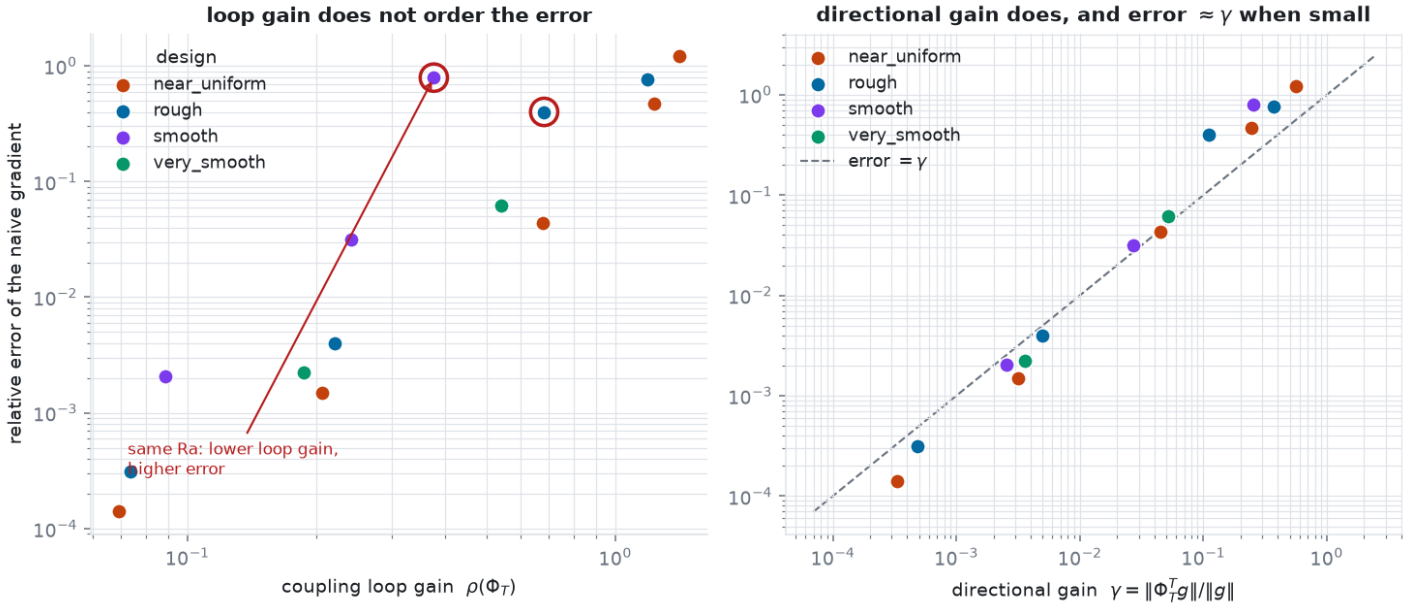
objective	outlet	top-half	chip peak	chip mean	domain	left column
γ	0.0027	0.0076	0.0261	0.0300	0.0967	0.3879
naive error	0.0117	0.0026	0.0385	0.0399	0.0211	0.3535

ρ(Φ_T) = 0.5481 on every column while the error varies **136-fold**. A constant cannot explain that spread: spectral radius alone is insufficient for objective-specific gradient error.

The directional gain. Equation (2) says the loop-cut adjoint has the exact residual $\Phi_T^{\mathrm{T}} g$, so the natural relative measure is

$$\gamma = \|\Phi_T^{\mathrm{T}}\, g\| \,/\, \|g\|, \quad (3)$$

one VJP through the loop. Across **14 converged configurations** drawn from four design families and five attempted Rayleigh levels, log γ correlates with log error at **0.995**, against 0.825 for ρ (`predict_error.py`). It remains 0.994–0.997 when each family is held out, with a seeded bootstrap 95% interval of 0.989–0.999 (`predictor_statistics.py`).



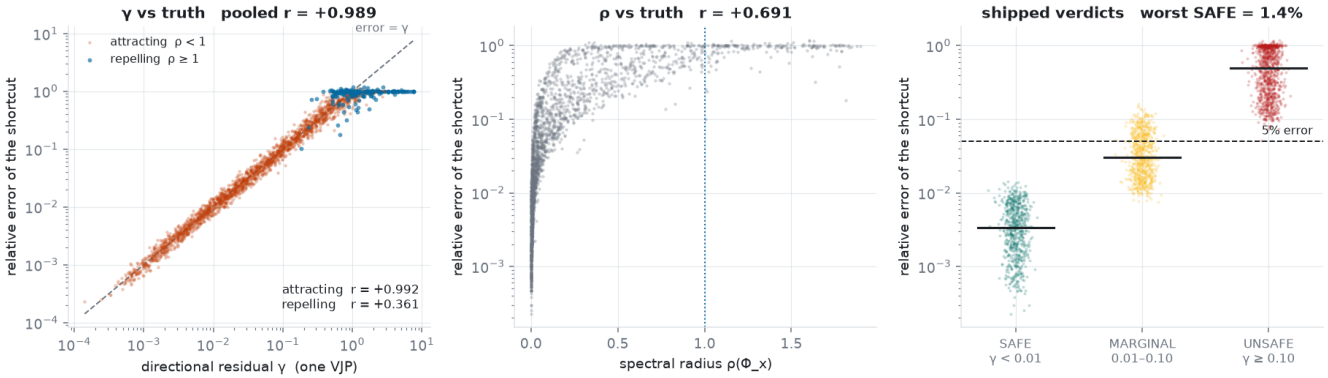
The directional gain predicts; the spectral radius does not. Each point is one (design, Rayleigh number) configuration, coloured by design family. Left: ρ(Φ_T) against the measured relative error of the component-wise gradient — log-log correlation 0.825, with visible order reversals, pairs where the configuration ρ calls safer is in fact the more damaged one. Right: γ against the same errors, correlation 0.995, tracking the dashed error = γ line while γ is small.

γ is a guide, not a formula. Across objectives it correlates at 0.80 and misses two rows above by about 4× in each direction — expected, since converting an adjoint residual to design-gradient error also depends on $(I-\Phi\_T^T)^{-1}$ and on Φ_θ^T. On this benchmark γ < 0.01 accompanied roughly percent-level error and γ > 0.1 flagged danger. `coupling_check.py` exposes these as configurable, benchmark-calibrated defaults rather than universal guarantees.

Does it generalise? Every number above comes from one physical system, which is the honest limit of the evidence. So we removed the physics: 2,377 randomly generated coupled fixed points (`gamma_generalization.py`) across four structural families — symmetric, non-normal, sparse, low-rank — with linear loops $\Phi = Ax + B\theta$ and nonlinear ones $\Phi = \tanh(Ax) + b$, spectral radius swept log-uniformly over 10^{-3} to 1.9, and every quantity available in closed form. γ is computed by calling the shipped module, not a reimplementation. Pooled, log γ correlates with log error at **0.989** against **0.691** for ρ, and γ wins in **every family and both kinds** — 0.996 symmetric, 0.982 non-normal, 0.995 sparse, 0.990 low-rank. The shipped thresholds survive contact: of 656 draws called SAFE the worst error was 1.4% and none exceeded 5%, and all 965 called UNSAFE genuinely exceeded it.

The same study locates the boundary, which we would rather not have found. Split by spectral radius, γ correlates 0.993 for attracting fixed points and only **0.36 for repelling ones**. This follows from (2): γ is a residual, and the error it induces is $(I - \Phi\_T^T)^{-1}$ applied to it — an amplification bounded by about $1/(1-\rho)$ only while $\rho < 1$. Correcting γ by the observed decay of $\|(\Phi\_T^T)^k g\|$ does not repair it (0.25). But the terms stop decaying in 136 of 178 repelling draws, which is the actionable signal: for a repelling loop, trust γ's *verdict* and not its *magnitude*, and compute the adjoint. Our own headline state is repelling at ρ = 1.19, and there we do.

One VJP predicts the cost of cutting a coupling loop, on 2,377 random coupled systems with no physics in them — and stops predicting when the fixed point repels



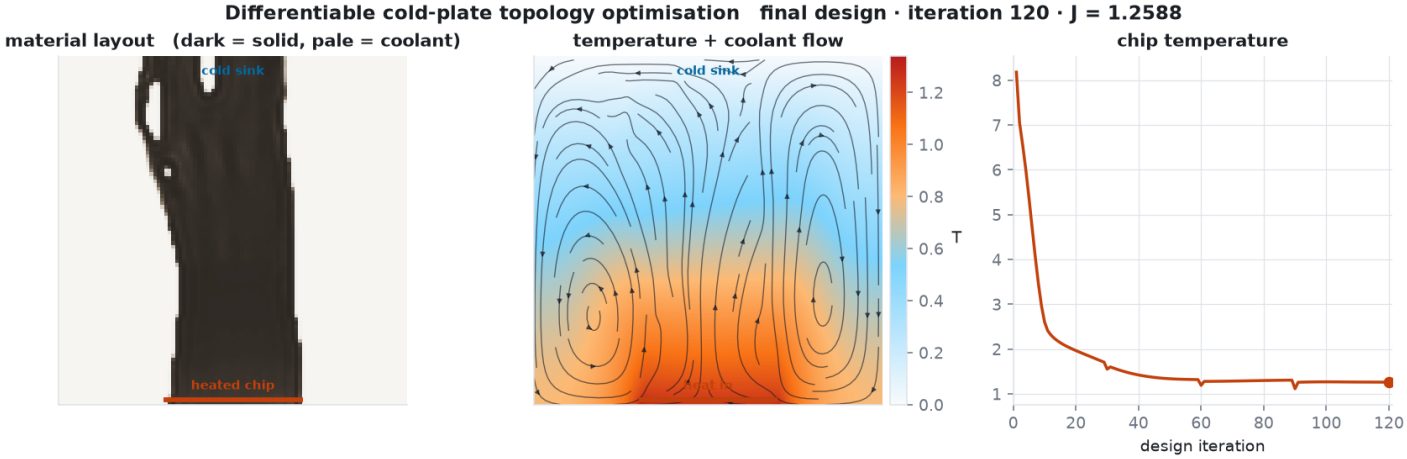
One VJP, no physics. Left: measured relative error of the component-wise gradient against γ on 2,377 randomly generated coupled fixed points, spanning four decades and hugging the identity error = γ; the repelling cases (blue) peel away and saturate, which is the documented limit. Centre: the spectral radius against the same errors — the obvious diagnostic, and visibly not a function of the thing it is meant to predict. Right: the thresholds this repository ships, with medians marked. Nothing γ called SAFE exceeded 1.4% error, and everything it called UNSAFE genuinely exceeded 5%.

Using it as a budget. Because γ costs one VJP against the tens the adjoint needs, it can be measured *before* deciding whether to pay. Gating the optimiser on it (`--mode gamma_gated`, 48^2, 80 iterations, gate 0.10) gives $\gamma \in [0.020, 0.074]$ throughout: the exact adjoint is never purchased, cross-boundary VJPs fall from 1015 to 80, and the design is unchanged (final J 1.3113 against 1.3180, 0.5% apart). At the Ra = 3×10^4 state of Section 7 the same gate returns γ = 0.404 and refuses, against a measured error of 115%; the repeated VJP norms are (0.404, 0.211, 0.145, 0.153). They are diagnostics, not a convergent Neumann series at this repelling fixed point.

The reading that matters is not the speed-up. It is that the two regimes are indistinguishable in J, in the residual, and in the convergence history, and that one VJP screens them — a VJP obtainable only by differentiating *through* the loop, which is to say by the same composition the gate may decline to use.

6. The design problem

Minimising mean chip temperature subject to a 35% solid-material budget, with density filtering and Heaviside continuation, 120 iterations at 96×96: **J falls from 8.18 to 1.26, an 84.6% reduction**. The optimiser finds a branching tree that conducts heat toward the sink while leaving channels open for buoyancy to carry the remainder — the structure the natural-convection topology optimisation literature reports.



The converged cold plate. Material layout (dark = solid metal), the resulting temperature field with streamlines of the buoyancy-driven flow, and the objective history. The branching conductor reaches from the chip strip on the bottom wall toward the cold sink while leaving the convection cells room to circulate; both are needed, and the balance between them is what the coupled gradient is resolving.

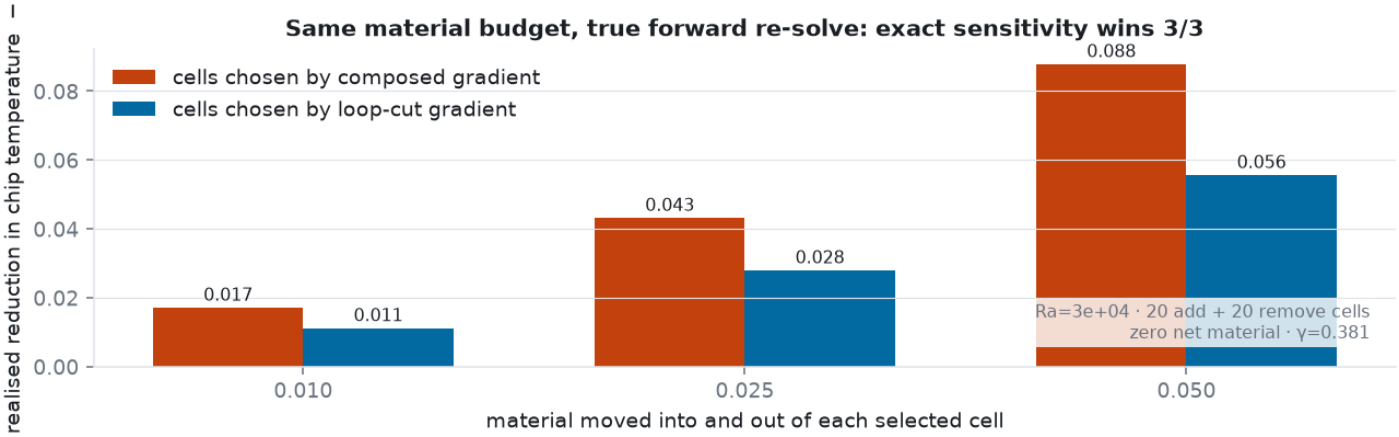
7. What the gradient changes — and what it does not

We ran the long optimisation twice, once with each gradient, and **both succeeded**; the shortcut finished marginally lower (1.2576 against 1.2588). Adam normalises each coordinate, and along this weakly coupled trajectory the shortcut's cosine stays above 0.98. A successful optimiser is not evidence that its gradient is right.

We therefore tested a discrete engineering action at the strong state instead (`intervention_test.py`, 20^2, Ra = 3×10^4). Each gradient receives the same zero-net-material budget: add material to its twenty most favourable cells and remove it from its twenty least favourable. We then discard both linear predictions and re-solve the true coupled forward problem:

material step	ΔJ, composed choice	ΔJ, shortcut choice	more cooling
0.010	−0.01715	−0.01121	53%
0.025	−0.04322	−0.02800	54%
0.050	**−0.08789**	−0.05565	**58%**

The gradients agree on only 40% of the add set and 10% of the remove set. The composed choice wins all three fresh forward solves, delivering **58% more realised cooling** at the largest step for exactly the same material budget.



Equal-budget actions checked by fresh coupled solves: the composed choice wins 3/3.

At fixed Ra = 2×10^4 and step 0.025 we then swept a seed range **declared before running** — 0 to 11, with losses recordable and non-convergence reported rather than dropped. Ten of the twelve designs had a reachable steady state, and **the composed choice won all ten**, median 36% extra cooling, range 6% to 276%. The design of that sweep is part of the result: its first version hand-picked its seeds and raised on a loss, so it could not have reported one.

The same distinction appears in attribution (`sensitivity_ranking.py`, 32^2, Ra = 3×10^4). The shortcut keeps the sign of all fifty truly most influential cells—enough for descent—but its ranking is chance-level (Spearman −0.011), it misses 44% of the true top fifty, and promotes a cell truly ranked 1016th of 1024. A serviceable search direction can still be a useless sensitivity.

Limitations are important. The intervention evidence spans one state over three amplitudes plus twelve pre-declared designs at a second state — a sweep, not a population study, and both states are strongly coupled by construction. The physics is two-dimensional; the fluid block carries the convective term when asked (Section 2), but the headline results are computed in the Stokes limit, which Section 2 justifies by measurement rather than assertion. The topology optimisation runs at Ra = 10^3 because its near-uniform intermediate-density start has no reachable steady state at the strong setting. Finally, γ is measured on one *physical* system plus 2,377 synthetic ones; the synthetic study establishes that the relationship is not an artefact of this problem, but random operators are not a substitute for a second real multiphysics application, and the repelling regime remains a stated exclusion rather than a solved case.

8. Reproducibility

Four pinned implementations, three served per run; 61 component tests; and a scheduled job that exercises the real container boundary. The safe reviewer path is `scripts/judge_demo.sh`; every experiment has a driver, and `scripts/audit_claims.py` re-derives the headline numbers.